

MAPPING TEXT TO PHONES FOR TTS

Module 3

- ▣ Contains list of words and its pronunciation
- ▣ Homographs (wind)
- ▣ TTS dictionaries dont represent dialectal variation
- ▣ ASR dictionary needs to represent both pronunciations of either and tomato, TTS can choose one.

Word	Pronunciation	Word	Pronunciation
cat	[kæt]	goose	[gʊs]
cats	[kæts]	geese	[giːs]
pig	[pɪɡ]	hedgehog	[ˈhedʒ.hɒɡ]
pigs	[pɪgz]	hedgehogs	[ˈhedʒ.hɒɡz]
fox	[fɒks]		
foxes	[ˈfɒksɪz]		

- ▣ Three large commonly used online pronunciation dictionaries (in both speech synthesis and recognition)
- ▣ PRONLEX
 - Designed for speech recognition
 - Contains 90,694 wordforms
 - All the words in WallStreet Journal & Switchboard Corpus
- ▣ CMUdict
 - Developed for ASR purposes
 - Has pronunciations abt 100,000 wordforms
- ▣ CELEX
 - Contains all words in Oxford Advanced Learners Dictionary and Longman Dictionary : 160,585 wordforms
 - British pronunciation , while other two are American
 - Derived from IPA while other two from ARPAbet

- ▣ Cases when we cant rely a dictionary
 - Names, Morphological productivity, Numbers
- ▣ 1. Names
 - It is difficult to list all proper names in Advance
 - It may come any language ,may have variable spellings
 - Most applications of TTS or ASR involve names

2.Morphology

- The possessive 's and plural s suffix are pronounced differently.
- (Gallo's,IBM's DATs,Syntex's v/s faxes,indies)
- faxes is pronounced [fæksiz], IBM's is pronounced [aIbijEmz], and DATs is pronounced [dæts]

3. Ways to pronounce string of digits

- **Serial:** each digit is pronounced separately — 8765 is “eight seven six five”
- **Combined:** the digit string is pronounced as a single integer, with all position labels read out — “eight thousand seven hundred sixty five”
- **Paired:** each pair of digits is pronounced as an integer; if there is an odd number of digits the first one is pronounced by itself — “eighty-seven sixty-five”.
- **Hundreds:** strings of four digits can be pronounced as counts of hundreds — “eighty-seven hundred (and) sixty-five”
- **Trailing Unit:** strings that end in zeros are pronounced serially until the last nonzero digit, which is pronounced followed by the appropriate unit — 8765000 is “eight seven six five thousand”.

An FST-based pronunciation lexicon

Fragment	Pronunciation
-p-	[p]
-ph-	[f]
-phe-	[fi]
-phes-	[fiz]
-place-	[pleis]
-placi-	[pleisi]
-plement-	[pliment]

Orthographic Lexicon	Lexicon
<i>Regular Nouns</i>	
cat	c k a æ t t
fox	f f o ɑ x ks
dog	d d o ɑ g g
<i>Irregular Singular Nouns</i>	
goose	g g oo u s s e ε
<i>Irregular Plural Nouns</i>	
g o:e o:e s e	g g oo u:ee i s s e ε

Figure 4.21 FST-based lexicon, extending the lexicon in the table on page 74 in Chapter 3. Each symbol in the lexicon is now a pair of symbols separated by '|', one representing the 'orthographic' lexical entry and one the 'phonological' lexical entry. The irregular plural *geese* also pre-specifies the contents of the intermediate tape ':ee|i'.

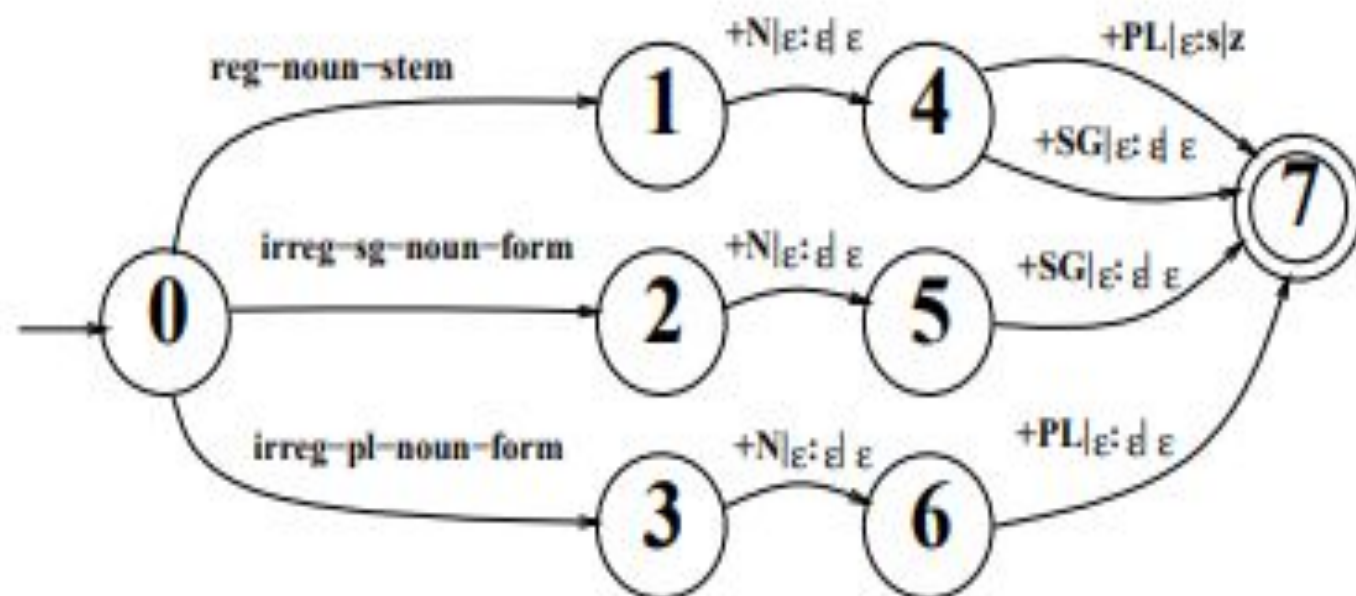


Figure 4.22 FST for the nominal singular and plural inflection. The automaton adds the morphological features [+N], [+PL], and [+SG] at the lexical level where relevant, and also adds the plural suffix *s|z* (at the intermediate level). We will discuss below why we represent the pronunciation of *-s* as *z* rather than *s*.

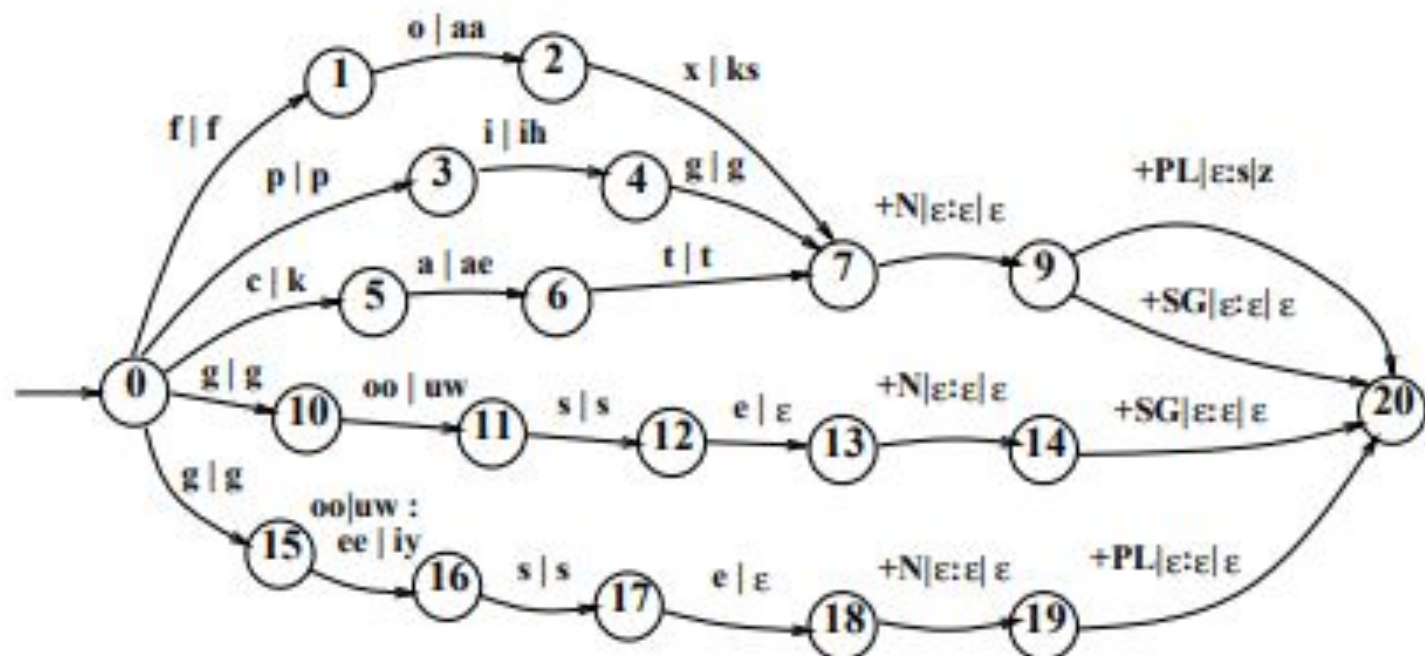


Figure 4.23 Mini-lexicon composing a transducer from the baseform lexicon of Figure 4.21 with the inflectional transducer of Figure 4.22.

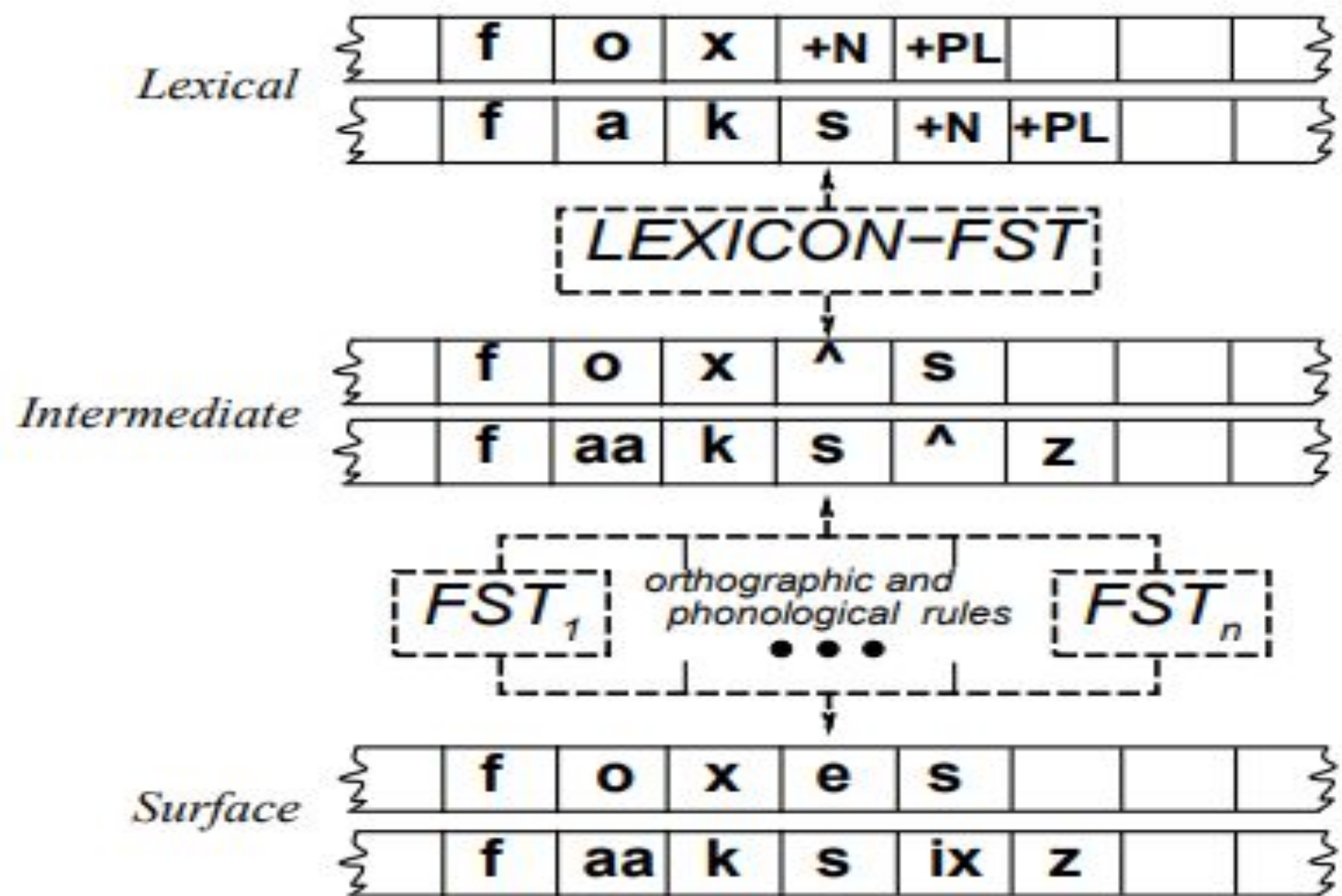


Figure 4.24 Mapping between the lexicon and surface form for orthography and phonology simultaneously. The system can be used to map from a lexical entry to its surface pronunciation or from surface orthography to surface pronunciation via the lexical entry.

Models of Pronunciation and Spelling

- ▣ ambiguity problem is heightened by **pronunciation variation**;
- ▣ for example the word *the* is sometimes pronounced THEE and sometimes THUH;
- ▣ High-quality speech synthesis algorithms need to know when to use particular pronunciation variants

- Given the sequence of letters corresponding to a mis-spelled word, we need to produce an ordered list of possible correct words.
- example the sequence *acress* might be a mis-spelling of *actress*, or of *cress*, or of *acres*.
- We transduce from the 'surface' form *acress* to the various possible 'lexical' forms, assigning each with a probability;
- we then select the most probable correct word.

DEALING WITH SPELLING ERRORS

- **Optical character recognition** is the term used for automatic recognition of machine or hand-printed characters.
- An optical scanner converts a machine or hand-printed page into a bitmap which is then passed to an OCR algorithm.
- **On-line handwriting recognition** is the recognition of human printed or cursive handwriting as the user is writing.
- Dynamic input- number and order of strokes, speed and direction of each stroke

- OCR systems often misread 'D' as 'O' or 'ri' as 'n', producing 'mis-spelled' words like *dension* for *derision*, or *POQ Bach* for *PDQ Bach*
- spelling correction, breaks the field down into three increasingly broader problems:

1. non-word error detection: detecting spelling errors which result in non-words (like *graffe* for *giraffe*).

2. isolated-word error correction: correcting spelling errors which result in non-words, for example correcting *graffe* to *giraffe*, but looking only at the word in isolation.

3. context-dependent error detection and correction: Using the context to help detect and correct spelling errors even if they accidentally result in an actual word of English (**real word errors**).

Eg: there or three, (typographic)

dessert for desert, piece for peace (wrong spelling)

SPELLING ERROR PATTERNS

- ▣ **insertion:** mistyping *the* as *ther*
- ▣ **deletion:** mistyping *the* as *th*
- ▣ **substitution:** mistyping *the* as *thw*
- ▣ **transposition:** mistyping *the* as *the*
- ▣ *Human typing errors*
 - *Typographic errors* (misspelling *spell* as *speel*)
 - *Cognitive errors* (misspelling *separate* as *seperate*)

- ▣ **Correct:**

The quick brown fox jumps over the lazy dog.

- ▣ **Recognized:**

'lhe q~ ick brown foxjurnps ovcr tb l azy dog.

- ▣ Substitutions (e->c)

- ▣ Mutisubstitutions (T -> 'l, m -> rn ,he->b)

- ▣ **Framing errors**

Probabilistic Models

- Bayesian inference
- Noisy channel
- problem of ASR pronunciation modeling, and the problem of spelling correction , can be modeled as problems of mapping from one string of symbols to another
- given a string of symbols representing the pronunciation of a word in context, we need to figure out the string of symbols representing the lexical or dictionary pronunciation
- given the incorrect sequence of letters in a mis-spelled word, we need to figure out the correct sequence of letters in the correctly

Noisy channel Model

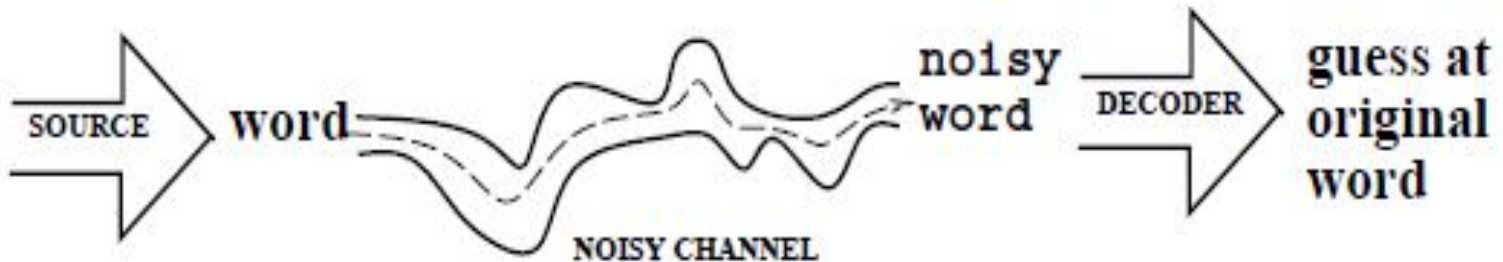


Figure 5.1 The noisy channel model

Noise : spelling errors which mask the correct spelling of the word.

Sources of noise

1. variation in pronunciation
2. variation in the realization of phones
3. acoustic variation due to the channel (microphones, telephone networks, etc)

Bayes inference model – pronunciation example

- ▣ as in any classification task, we are given some observation and our job is to determine which of a set of classes it belongs to
- ▣ For speech recognition, the observation is the string of phones which make up a word.
- ▣ For spelling error detection, the observation might be the string of letters that constitute a possibly-misspelled word.
- ▣ in the speech case, no matter which of the many possible ways the word about is pronounced ,we want to classify it as about.
- ▣ In the spelling case, no matter how the word separate is misspelled, we'd like to recognize it as separate

Bayes inference model – pronunciation example

$$\hat{w} = \operatorname{argmax}_{w \in V} P(w|O)$$

$$P(x|y) = \frac{P(y|x)P(x)}{P(y)}$$

$$\hat{w} = \operatorname{argmax}_{w \in V} \frac{P(O|w)P(w)}{P(O)}$$

Key Concept #3. $\hat{w} = \operatorname{argmax}_{w \in V} \overbrace{P(O|w)}^{\text{likelihood}} \overbrace{P(w)}^{\text{prior}}$

- We are given a string of phones (say [ni]). We want to know which word corresponds to this string of phones
- Out of this universe of words, we want to choose the word which is most probable given the observation we have ([ni]).
- In other words, we want, out of all words in the vocabulary V the single word such that $P(\text{word}/\text{observation})$ is highest.

- ▣ you might be interested in finding out a patient's probability of having liver disease if they are an alcoholic
- ▣ **A** could mean the event "Patient has liver disease." Past data tells you that 10% of patients entering your clinic have liver disease. $P(A) = 0.10$.
- ▣ **B** could mean the litmus test that "Patient is an alcoholic." Five percent of the clinic's patients are alcoholics. $P(B) = 0.05$.
- ▣ You might also know that among those patients diagnosed with liver disease, 7% are alcoholics. This is your **B | A**: the probability that a patient is alcoholic, given that they have liver disease, is 7%.

- ▣ Bayes' theorem tells you:
$$P(A | B) = (0.07 * 0.1)/0.05 = 0.14$$

APPLYING THE BAYESIAN METHOD TO SPELLING

- ▣ algorithm to correct misspelled word *acress*
- ▣ The algorithm has two stages:
 - ▣ *Proposing candidate corrections*
 - ▣ *Scoring the candidates.*
- ▣ to propose candidate corrections Kernighan et al. make the simplifying assumption that the **correct word will differ from the misspelling by a single insertion, deletion, substitution, or transposition**

- Applying all possible transformations to *acress* yields the list of candidate words in Figure 5.2.

Error	Correction	Transformation			
		Correct Letter	Error Letter	Position (Letter #)	Type
acress	actress	t	—	2	deletion
acress	cress	—	a	0	insertion
acress	caress	ca	ac	0	transposition
acress	access	c	r	2	substitution
acress	across	o	e	3	substitution
acress	acres	—	2	5	insertion
acress	acres	—	2	4	insertion

Figure 5.2 Candidate corrections for the misspelling *acress*, together with the transformations that would have produced the error, after Kernighan *et al.* (1990). ‘—’ represents a null letter.

- The second stage of the algorithm scores each correction .
- Let t represent the typo (the misspelled word), and let c range over the set C of candidate corrections
- $C = \operatorname{argmax}_c p(t | c) p(c)$
- $P(c)$ can be estimated by counting how often the word c occurs in some corpus, and then **normalizing** these counts by the total count of all words

- ▣ $P(c) = \text{count}(c)/N$
- ▣ Zero counts can cause problems, and so we will add .5 to all the counts. This is called 'smoothing'

$$P(c) = \frac{C(c) + 0.5}{N + 0.5V}$$

- ▣ AP newswire corpus : N is 44 million. Since in this corpus, the word actress occurs 1343 times, the word acres 2879 times, and so on, the resulting prior probabilities are as follows:

c	freq(c)	p(c)
actress	1343	.0000315
cress	0	.000000014
caress	4	.0000001
access	2280	.000058
across	8436	.00019
acres	2879	.000065

Confusion Matrix

- a square 26x26 table which represents the number of times one letter was incorrectly used instead of another.
- the cell labeled [o, e] in a substitution confusion matrix would give the count of times that e was substituted for o.
- The cell labeled [t , s] in an insertion confusion matrix would give the count of times that t was inserted after s

Confusion Matrix

- ▣ $\text{del} [x, y]$ contains the number of times in the training set that the characters xy in the correct word were typed as x .
- ▣ $\text{ins} [x, y]$ contains the number of times in the training set that the character x in the correct word was typed as xy .
- ▣ $\text{sub} [x, y]$ the number of times that x was typed as y .
- ▣ $\text{trans} [x, y]$ the number of times that xy was typed as yx .

Formula to compute $p(t|c)$

$$P(t|c) = \begin{cases} \frac{\text{del}[c_{p-1}, c_p]}{\text{count}[c_{p-1}c_p]}, & \text{if deletion} \\ \frac{\text{ins}[c_{p-1}, t_p]}{\text{count}[c_{p-1}]}, & \text{if insertion} \\ \frac{\text{sub}[t_p, c_p]}{\text{count}[c_p]}, & \text{if substitution} \\ \frac{\text{trans}[c_p, c_{p+1}]}{\text{count}[c_p c_{p+1}]}, & \text{if transposition} \end{cases}$$

C_p is the p^{th} character of word c

c	freq(c)	p(c)	p(t c)	p(t c)p(c)	%
actress	1343	.0000315	.000117	3.69×10^{-9}	37%
cress	0	.000000014	.00000144	2.02×10^{-14}	0%
caress	4	.0000001	.00000164	1.64×10^{-13}	0%
access	2280	.000058	.000000209	1.21×10^{-11}	0%
across	8436	.00019	.0000093	1.77×10^{-9}	18%
acres	2879	.000065	.0000321	2.09×10^{-9}	21%
acres	2879	.000065	.0000342	2.22×10^{-9}	23%

Figure 5.3 Computation of the ranking for each candidate correction. Note that the highest ranked word is not *actress* but *acres* (the two lines at the bottom of the table), since *acres* can be generated in two ways. The *del[]*, *ins[]*, *sub[]*, and *trans[]* confusion matrices are given in full in Kernighan *et al.* (1990).

Minimum edit distance

- Kernighan et. al assumes that each word had only a single spelling error
- A more powerful algorithm which could handle the case of multiple errors, based on String Distance
- The minimum edit distance between two strings is the minimum number of editing operations (insertion, deletion, substitution) needed to transform one string into another

Minimum edit distance

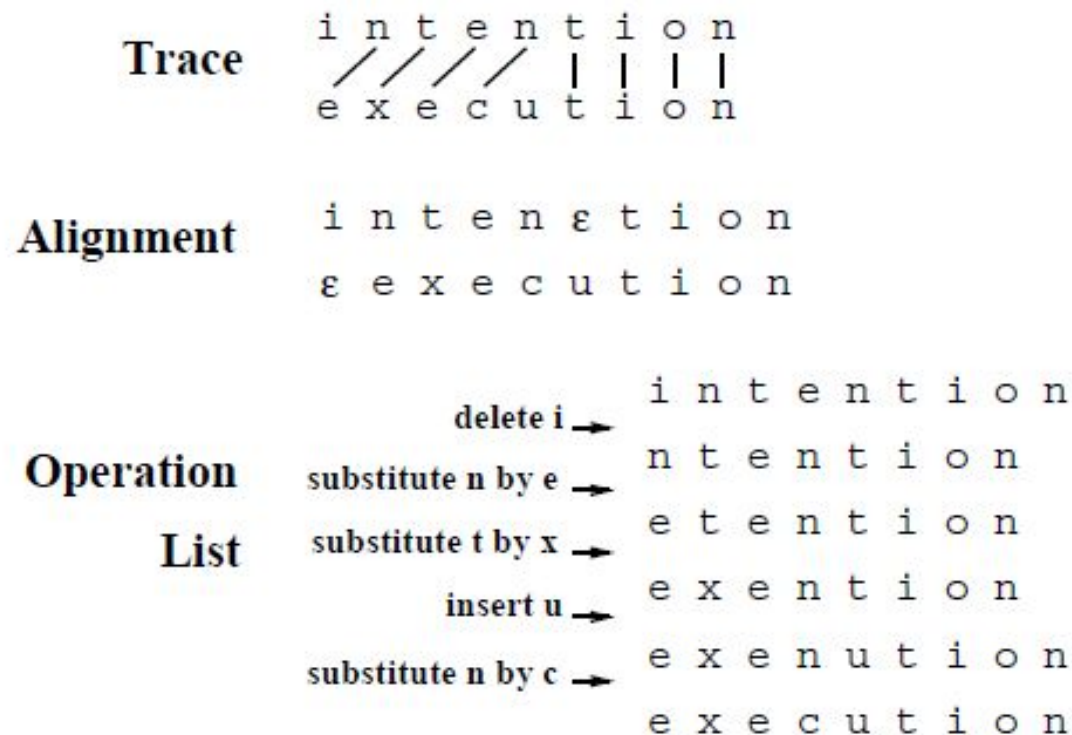


Figure 5.4 Three methods for representing differences between sequences (after Kruskal (1983))

Key concept

$$P(t|c) = \min \begin{cases} distance[i-1, j] + ins-cost(target_j) \\ distance[i-1, j-1] + subst-cost(source_j, target_i) \\ distance[i, j-1] + ins-cost(source_j) \end{cases}$$

Min Edit Distance Algorithm

function MIN-EDIT-DISTANCE(*target*, *source*) **returns** *min-distance*

$n \leftarrow \text{LENGTH}(\text{target})$

$m \leftarrow \text{LENGTH}(\text{source})$

Create a distance matrix $\text{distance}[n+1, m+1]$

$\text{distance}[0, 0] \leftarrow 0$

for each column i **from** 0 to n **do**

for each row j **from** 0 to m **do**

$\text{distance}[i, j] \leftarrow \text{MIN}(\text{distance}[i-1, j] + \text{ins-cost}(\text{target}_j),$
 $\text{distance}[i-1, j-1] + \text{subst-cost}(\text{source}_j, \text{target}_i),$
 $\text{distance}[i, j-1] + \text{ins-cost}(\text{source}_j))$

Ins cost =1
 Del cost =1
 Sub cost=2

Worked Example

n	9	10	11	10	11	12	11	10	9	8
o	8	9	10	9	10	11	10	9	8	9
i	7	8	9	8	9	10	9	8	9	10
t	6	7	8	7	8	9	8	9	10	11
n	5	6	7	6	7	8	9	10	11	12
e	4	5	6	5	6	7	8	9	10	11
t	3	4	5	6	7	8	9	10	11	12
n	2	3	4	5	6	7	8	8	10	11
i	1	2	3	4	5	6	7	8	9	10
#	0	1	2	3	4	5	6	7	8	9
	#	e	x	e	c	u	t	i	o	n

Figure 5.6 Computation of minimum edit distance between *intention* and *execution* via algorithm of Figure 5.5, using Levenshtein distance with cost of 1 for insertions or deletions, 2 for substitutions. Substitution of a character for itself has a cost of 0.

The Edit Distance Table

N	9	8	9	10	11	12	11	10	9	8
O	8	7	8	9	10	11	10	9	8	9
I	7	6	7	8	9	10	9	8	9	10
T	6	5	6	7	8	9	8	9	10	11
N	5	4	5	6	7	8	9	10	11	10
E	4	3	4	5	6	7	8	9	10	9
T	3	4	5	6	7	8	7	8	9	8
N	2	3	4	5	6	7	8	7	8	7
I	1	2	3	4	5	6	7	6	7	8
#	0	1	2	3	4	5	6	7	8	9
	#	E	X	E	C	U	T	I	O	N

Ins cost =1
 Del cost =1
 Sub cost = {0, if
 $i=j$,
 2 if $i \neq j$ }

-	0	E	X	E	C	U	T	I	O	N
0	0	1	2	3	4	5	6	7	8	9
I	1	1	2	3	4	5	6	6	7	8
N	2	2	2	3	4	5	6	7	7	7
T	3	3	3	3	4	5	5	6	7	8
E	4	3	4	3	4	5	6	6	7	8
N	5	4	4	4	4	5	6	7	7	7
T	6	5	5	5	5	5	5	6	7	8
I	7	6	6	6	6	6	6	5	6	7
O	8	7	7	7	7	7	7	6	5	6
N	9	8	8	8	8	8	8	7	6	5

Ins cost =1
 Del cost =1
 Sub cost={0, if
 $i=j$, 1 if $i \neq j$ }